Introducción a la Programación Orientada a Objetos

Sesión 2: clases, objetos, atributos e instancias

Lo que veremos hoy

1 ¿Qué es la POO?

Tema 2.1: el problema de las variables sueltas

2 Objetos

Tema 2.2: estado, comportamiento e identidad

3 Clases

Tema 2.3: el molde y su sintaxis en Java

4 Atributos

Tema 2.4: tipo, nombre y valores por defecto

5 Instancias y referencias

Temas 2.5 y 2.6: new, el punto y la memoria

6 Métodos y los 4 pilares

Abstracción, encapsulamiento, herencia y polimorfismo

Después de cada tema hay una práctica corta: primero se explica, luego se practica.

El problema: 5 estudiantes con variables sueltas

```
String nombre1 = "Ana";     double promedio1 = 8.7;
String nombre2 = "Luis";    double promedio2 = 9.1;
String nombre3 = "María";   double promedio3 = 7.5;
String nombre4 = "Pedro";   double promedio4 = 6.8;
String nombre5 = "Sofía";   double promedio5 = 9.4;

System.out.println(nombre1 + " : " + promedio1);
System.out.println(nombre2 + " : " + promedio2);
// ... y así hasta nombre5
```

10 variables para guardar solo nombre y promedio de 5 estudiantes

15 variables si además queremos guardar la edad

Nada une a nombre1 con promedio1, solo el número que les pusimos. ¿Y si cada estudiante fuera un solo “paquete” que guarda su nombre y su promedio juntos? Esa es la idea de la POO.

¿Qué es la Programación Orientada a Objetos?

La **Programación Orientada a Objetos (POO)** es un paradigma de programación que organiza el programa en **objetos**: cada objeto junta sus datos (atributos) y las acciones que puede hacer (métodos), igual que las cosas de la vida real.

Programación estructurada

- Los datos y las funciones van por separado.
- El programa es una secuencia de pasos.
- Lo que hiciste en la sesión 1: variables, if y for.

Programación orientada a objetos

- Los datos y las acciones van juntos dentro de cada objeto.
- El programa son objetos que colaboran entre sí.
- Se modela el problema como en la vida real.

Lenguajes orientados a objetos: Java, C#, C++, Python y Kotlin, entre otros.

Práctica 2.1: variables sueltas

1

Ejercicio

Escribe un programa con variables sueltas (nombre1, promedio1 … nombre5, promedio5) que muestre los 5 estudiantes uno debajo de otro con el formato nombre : promedio.

2

Continúa el ejercicio anterior

Agrégale la edad de cada estudiante en la misma línea. Cuenta cuántas variables y líneas nuevas hicieron falta y explica por qué esta forma no escala.

En la computadora y sin IA. Antes de ejecutar, escribe la salida que esperas.

Cómo se piensa en objetos

Atributos: cómo es

- color: rojo
- marca: Chevrolet
- kilometraje: 12000 km
- velocidad: 0 km/h



Un auto

Métodos: qué hace

- `encender()`
- `acelerar()`
- `frenar()`
- `apagar()`

En la vida real describimos cualquier cosa así: por cómo es y por lo que puede hacer. La POO lleva esa misma idea al código.

Objeto

Un **objeto** es un elemento concreto que existe dentro del programa. Todo objeto tiene un **estado**, un **comportamiento** y una **identidad** propia.



Estado

Los valores que tienen sus atributos en este momento.

Tu celular: negro, 128 GB, batería al 70 %.



Comportamiento

Las acciones que puede realizar: sus métodos.

Tu celular: llamar, enviar un mensaje, tomar una foto.



Identidad

Cada objeto es único, aunque otro tenga los mismos valores.

Tu celular no es el de tu compañero, aunque sean del mismo modelo.

Más objetos de la vida real: tu mochila, el auto de tu familia, la cuenta bancaria de Ana, el estudiante Luis.

Práctica 2.2: objetos del aula

1

Ejercicio

Elige 3 objetos del aula (una mochila, un celular, una silla) y escribe 4 características de cada uno con sus valores reales.

2

Continúa el ejercicio anterior

Busca otro objeto del mismo tipo que uno de ellos (otra mochila) y compara qué características comparten y qué valores cambian.

En parejas, en el cuaderno. Todavía no se usa la computadora.

Clase

Una **clase** es el molde o plano que describe qué atributos y qué métodos tendrán sus objetos. **La clase no es un objeto: solo lo describe.** A partir de una clase se pueden crear muchos objetos.



Molde de galletas

Clase: El molde

Objetos: Cada galleta



Plano de una casa

Clase: El plano

Objetos: Cada casa construida



Ficha de matrícula

Clase: La ficha en blanco

Objetos: Cada ficha llena de un estudiante

Con un molde no te comes nada: te comes las galletas. Con una clase tampoco guardas datos: los guardan sus objetos.

Clase vs objeto

Aspecto	Clase	Objeto
Qué es	Un molde: la descripción de cómo son y qué hacen	Un elemento concreto creado a partir del molde
Cuántos hay	Una sola por cada tipo (una clase Estudiante)	Todos los que se necesiten (Ana, Luis, María…)
Cuándo aparece	Se escribe en el código, en un archivo .java	Se crea al ejecutar el programa, con new
Datos	Dice qué atributos existen, sin valores de un caso real	Guarda sus propios valores en cada atributo
Ejemplo	`Estudiante`	`ana, luis, maria`

Regla rápida: si puedes preguntar "¿cuál?", es un objeto. Si preguntas "¿cómo son todos?", es una clase.

Una clase en Java

```java
public class Estudiante {
    // Atributos
    String nombre;
    int edad;
    double promedio;
    boolean matriculado;
}
```

La clase sola no hace nada visible: todavía no existe ningún estudiante en memoria. Para eso hay que crear instancias.

1. **`public class`**
   Palabras que declaran una clase pública.

2. **`Estudiante`**
   Nombre de la clase en PascalCase. El archivo debe llamarse Estudiante.java.

3. **`{ }`**
   Todo lo que pertenece a la clase va entre estas llaves.

4. **`String nombre;`**
   Cada atributo: tipo, nombre en camelCase y punto y coma.

Práctica 2.3: tu primera clase

1

Ejercicio

Crea en el IDE un proyecto con el archivo Estudiante.java que contenga la clase Estudiante y compílalo sin errores.

2

Continúa el ejercicio anterior

Agrega al mismo proyecto una clase Main con el método main que imprima "Proyecto listo" y ejecútalo.

En la computadora y sin IA. Antes de ejecutar, escribe la salida que esperas.

Atributos

Un **atributo** es una característica de un objeto. Se declara en la clase con un **tipo** y un **nombre**; cada objeto guarda **su propio valor** en ese atributo.

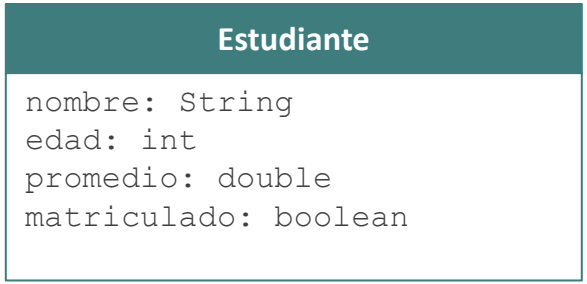


Diagrama de la clase: nombre arriba y atributos debajo, con su tipo.

Atributo	Tipo	Objeto ana	Objeto luis
`nombre`	String	"Ana"	"Luis"
`edad`	int	19	20
`promedio`	double	8.7	9.1
`matriculado`	boolean	true	false

Los dos objetos tienen los mismos atributos, pero cada uno con sus propios valores.

Valores por defecto de los atributos

Si a un atributo no se le asigna un valor, Java le pone uno por defecto según su tipo.

Tipo del atributo	Valor por defecto
int	0
double	0.0
boolean	false
char	'\u0000' (carácter nulo)
String (y cualquier objeto)	null (no apunta a nada)

nuevo : Estudiante

```
nombre = null
edad = 0
promedio = 0.0
matriculado = false
```

Así queda un estudiante al que todavía no se le asignó ningún valor. Lo comprobarás en el tema 2.5, cuando crees objetos con new.



Ojo: esto solo pasa con los atributos. Una variable local, por ejemplo dentro de main, debe tener un valor antes de usarse o el programa no compila.

Práctica 2.4: atributos

1

Ejercicio

Agrega a la clase Estudiante los atributos nombre (String) y promedio (double).

2

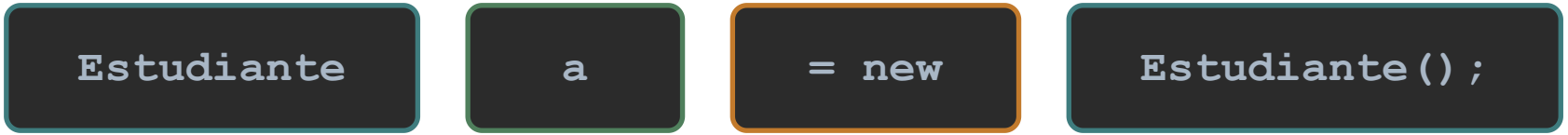
Continúa el ejercicio anterior

Agrega también edad (int) y aprobado (boolean), y escribe el valor por defecto de cada uno de los 4 atributos.

En la computadora y sin IA. Antes de ejecutar, escribe la salida que esperas.

Instancia: crear un objeto con new

Instanciar es crear un objeto a partir de una clase. El objeto creado se llama **instancia** de esa clase. Objeto e instancia son lo mismo: “instancia” resalta de qué clase salió.



Tipo de la variable: la clase

Nombre de la referencia

Crea el objeto en memoria

Constructor: arma el objeto nuevo

Se lee así: “crea un nuevo Estudiante y guárdalo en a”. Cada vez que se escribe `new` aparece un objeto distinto, aunque sea de la misma clase.

El operador punto

Con el **punto** se entra a un objeto para usar sus atributos y métodos: `objeto.atributo` o `objeto.metodo()`.

```
Estudiante a = new Estudiante();

// Asignar valores (escribir)
a.nombre = "Ana";
a.edad = 19;
a.promedio = 8.7;

// Leer valores
System.out.println(a.nombre + " : " + a.promedio);
```

Salida en consola

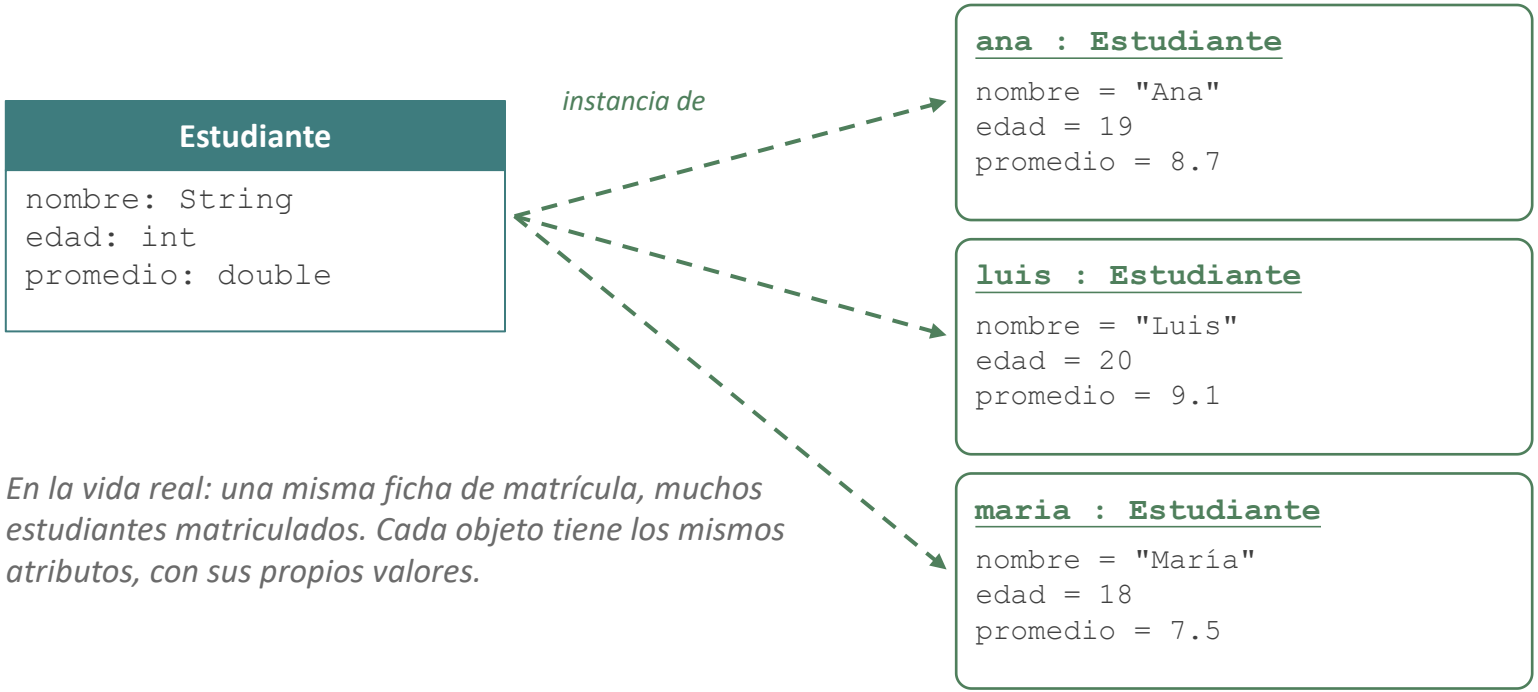
```
Ana : 8.7
```

`a.nombre = "Ana";`
Guarda "Ana" en el atributo nombre del objeto a.

`a.nombre`
Lee el valor que tiene ahora.

Es como decir “el nombre de a”, “la edad de a”.

Una clase, muchas instancias



En la vida real: una misma ficha de matrícula, muchos estudiantes matriculados. Cada objeto tiene los mismos atributos, con sus propios valores.

Resolviendo el problema de los 5 estudiantes

```
Estudiante e1 = new Estudiante();
e1.nombre = "Ana";    e1.promedio = 8.7;
Estudiante e2 = new Estudiante();
e2.nombre = "Luis";   e2.promedio = 9.1;
Estudiante e3 = new Estudiante();
e3.nombre = "María";  e3.promedio = 7.5;
Estudiante e4 = new Estudiante();
e4.nombre = "Pedro";  e4.promedio = 6.8;
Estudiante e5 = new Estudiante();
e5.nombre = "Sofía";  e5.promedio = 9.4;

System.out.println(e1.nombre + " : " + e1.promedio);
// ... igual para e2, e3, e4 y e5
```

Salida en consola

```
Ana : 8.7
Luis : 9.1
María : 7.5
Pedro : 6.8
Sofía : 9.4
```

Ahora el nombre y el promedio de cada estudiante viajan juntos dentro de un mismo objeto.

Si mañana se agrega la edad, se añade una sola línea en la clase y todos los objetos la tienen.

Práctica 2.5: instancias

1

Ejercicio

En main, crea una instancia de Estudiante, asígnale nombre y promedio e imprímela con el formato nombre : promedio.

2

Continúa el ejercicio anterior

Amplíalo a 5 instancias y muestra los 5 estudiantes uno debajo de otro, resolviendo el problema del tema 2.1.

En la computadora y sin IA. Antes de ejecutar, escribe la salida que esperas.

Referencias y objetos en memoria

```java
Estudiante a = new Estudiante();
a.promedio = 8.7;

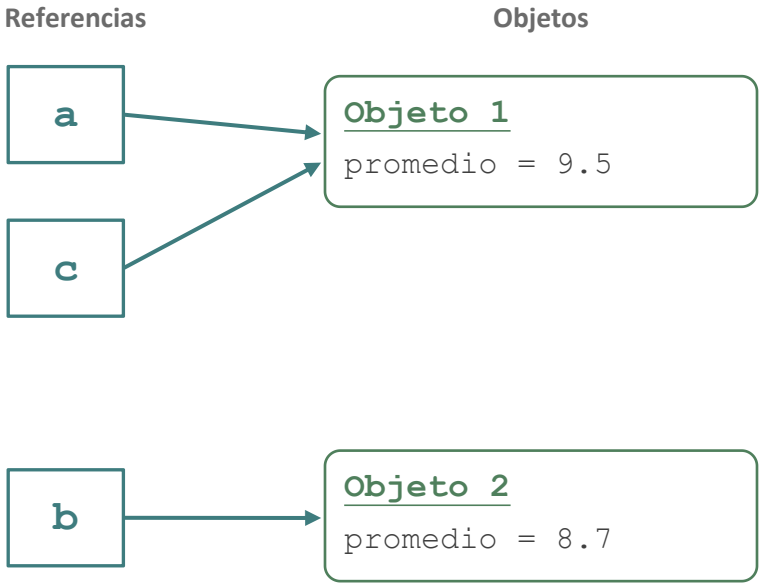
Estudiante b = new Estudiante();
b.promedio = 8.7;

Estudiante c = a;    // no usa new
c.promedio = 9.5;

System.out.println(a.promedio);
System.out.println(a == c);
System.out.println(a == b);
```

Salida

```
9.5
true
false
```



`c = a` no copia el objeto: copia la flecha. `a` y `c` apuntan al mismo objeto.

Práctica 2.6: memoria viva

1

Ejercicio

Con 2 estudiantes como objetos (a y b) y 2 como referencias, representa a.promedio = 9; y di qué cartel cambia.

2

Continúa el ejercicio anterior

Continúa con Estudiante c = a; y c.promedio = 6; y responde qué promedio tiene ahora a y por qué.

Actividad grupal en el aula, con carteles. Luego se comprueba en la computadora.

Métodos: lo que el objeto sabe hacer

Un **método** es una acción que puede realizar un objeto. Se escribe dentro de la clase y puede usar y cambiar los atributos de ese objeto.

```java
public class Auto {
    String marca;
    int velocidad;

    void acelerar(int cantidad) {
        velocidad = velocidad + cantidad;
    }

    void frenar() {
        velocidad = 0;
    }
}
```

```java
Auto miAuto = new Auto();
miAuto.marca = "Chevrolet";
miAuto.acelerar(40);
miAuto.acelerar(20);
System.out.println(miAuto.velocidad);
miAuto.frenar();
System.out.println(miAuto.velocidad);
```

Salida en consola

```
60
0
```

Los 4 pilares de la POO

Las ideas que hacen que la POO funcione



Abstracción



Encapsulamiento



Herencia



Polimorfismo

1. Abstracción

Abstraer es quedarse solo con las características y acciones que importan para resolver el problema, e ignorar los detalles que no hacen falta.

Cómo se usa

Al diseñar una clase decides qué atributos y qué métodos poner. Lo que no sirve al problema, no entra.

En la vida real

Usas el control remoto con sus botones sin saber cómo están hechos sus circuitos. Conduces un auto con volante y pedales sin conocer el motor por dentro.



El control remoto

Lo que necesitas ver:
encender, volumen, cambiar de canal

Lo que se esconde:
circuitos, señal infrarroja, código interno

Abstracción: el mismo objeto, distintos problemas

Una persona tiene cientos de características. Cada sistema se queda solo con las que necesita:

	Sistema académico	Consultorio médico	Gimnasio
Clase	`Estudiante`	`Paciente`	`Socio`
Atributos	nombre, cédula, carrera, promedio	nombre, cédula, tipo de sangre, alergias	nombre, peso, estatura, plan
Métodos	matricularse(), verNotas()	agendarCita(), verHistorial()	registrarIngreso(), pagarMensualidad()
No le interesa	tipo de sangre, peso	carrera, promedio	carrera, alergias

En Java la abstracción se ve en la clase que escribes: solo pones los atributos y métodos que el problema necesita.

2. Encapsulamiento

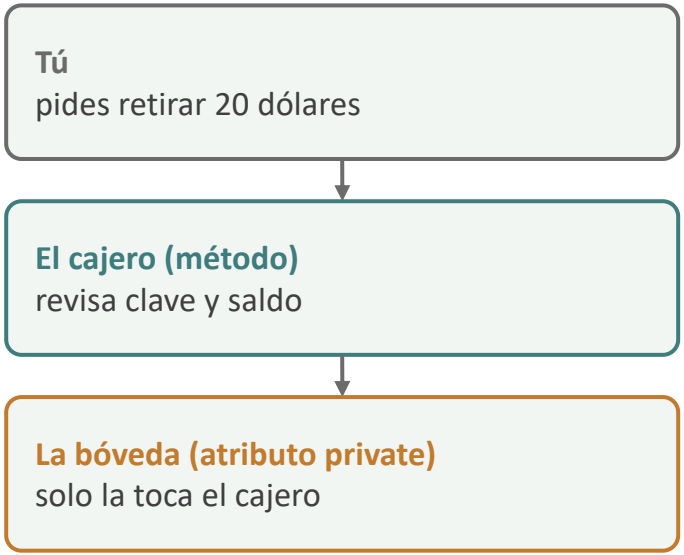
Encapsular es proteger los datos de un objeto para que solo se puedan leer o cambiar a través de sus métodos, que controlan que los cambios sean válidos.

Cómo se usa en Java

Los atributos se declaran `private` y se ofrecen métodos `public` para usarlos, por ejemplo `getSaldo()` para leer y `depositar()` para cambiar.

En la vida real

En un cajero automático no tocas el dinero de la bóveda: pides retirar y el cajero revisa tu clave y tu saldo antes de entregarte nada.



Encapsulamiento en Java

```java
public class CuentaBancaria {
    private double saldo;

    public void depositar(double monto) {
        if (monto > 0) {
            saldo = saldo + monto;
        }
    }

    public double getSaldo() {
        return saldo;
    }
}
```

```java
CuentaBancaria cuenta =
    new CuentaBancaria();
cuenta.depositar(100);
cuenta.depositar(-50); // se ignora
System.out.println(cuenta.getSaldo());
// cuenta.saldo = -500;
// ERROR: saldo has private access
```

Salida en consola

```
100.0
```

Modificador	Se puede usar
`private`	Solo en su clase
`public`	Desde cualquier clase

También existen protected y el acceso sin modificador, que se verán más adelante.

3. Herencia

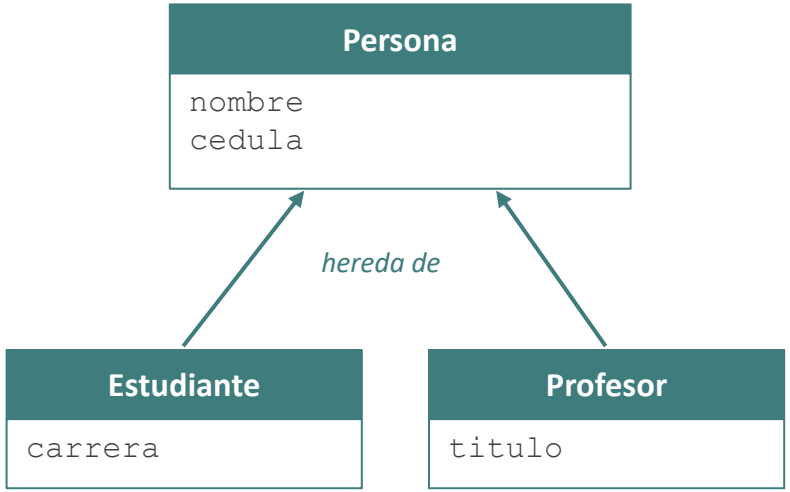
La **herencia** permite crear una clase nueva a partir de otra. La **clase hija** (subclase) recibe los atributos y métodos de la **clase padre** (superclase) y agrega los suyos.

Cómo se usa

Se usa cuando hay una relación “es un”: un Estudiante es una Persona. En Java se escribe con `extends`.

En la vida real

En la universidad, estudiantes y profesores son personas: todos tienen nombre y cédula. Además, el estudiante tiene carrera y el profesor tiene título.



Herencia en Java: extends

```java
public class Persona {
    String nombre;
    String cedula;

    void saludar() {
        System.out.println("Hola, soy " + nombre);
    }
}

public class Estudiante extends Persona {
    String carrera;
}
```

Cada clase pública va en su propio archivo .java.

```java
Estudiante e = new Estudiante();
e.nombre = "Ana";        // heredado
e.carrera = "Software"; // propio
e.saludar();             // heredado
```

Salida en consola

```
Hola, soy Ana
```

- En Java una clase solo puede heredar de una clase padre.
- Lo que es private en la clase padre no se usa directamente en la hija.
- Toda clase hereda de Object, aunque no se escriba.

4. Polimorfismo

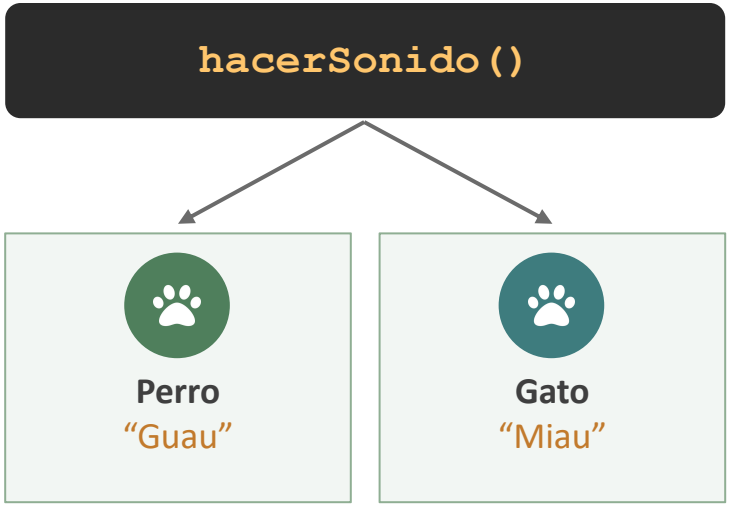
Polimorfismo significa “muchas formas”: el mismo mensaje, es decir, llamar al mismo método, produce un comportamiento distinto según el objeto que lo recibe.

Cómo se usa

La clase padre define un método y cada hija lo **sobrescribe** con su propia versión usando `@Override`.

En la vida real

Presionas el mismo botón “reproducir” y suena una canción, se ve un video o empieza un podcast: la orden es la misma, el resultado depende de qué se reproduce.



Polimorfismo en Java: sobrescritura

```java
public class Animal {
    void hacerSonido() {
        System.out.println("...");
    }
}

public class Perro extends Animal {
    @Override
    void hacerSonido() {
        System.out.println("Guau");
    }
}
// Gato igual, con "Miau"
```

```java
Animal a1 = new Perro();
Animal a2 = new Gato();
a1.hacerSonido();
a2.hacerSonido();
```

Salida en consola

```
Guau
Miau
```

También es polimorfismo: la sobrecarga.
Varios métodos con el mismo nombre y distintos parámetros en la misma clase, por ejemplo `area(double radio)` y `area(double base, double altura)`.

Los 4 pilares de un vistazo

Pilar	Idea clave	En Java	En la vida real
Abstracción	Quedarse solo con lo importante	Elegir los atributos y métodos de la clase	Control remoto
Encapsulamiento	Proteger los datos del objeto	`private + get / set`	Cajero automático
Herencia	Reutilizar una clase para crear otra	`extends`	Estudiante y Profesor son Persona
Polimorfismo	Mismo mensaje, distinto resultado	`@Override`	Botón reproducir

Los cuatro trabajan juntos: al diseñar una clase se abstrae, se encapsula, y luego se puede heredar y sobrescribir.

Práctica: los 4 pilares



Abstracción

Haz la lista de atributos de una clase Celular para una tienda que lo vende y para un servicio técnico que lo repara. Explica por qué cambian.



Encapsulamiento

Crea la clase Producto con precio private y un método setPrecio que no acepte valores negativos. Prueba con 2.50 y con -3.



Herencia

Crea la clase Vehiculo (marca, anio) y las hijas Moto (cilindraje) y Camion (capacidadCarga). Crea una instancia de cada una.



Polimorfismo

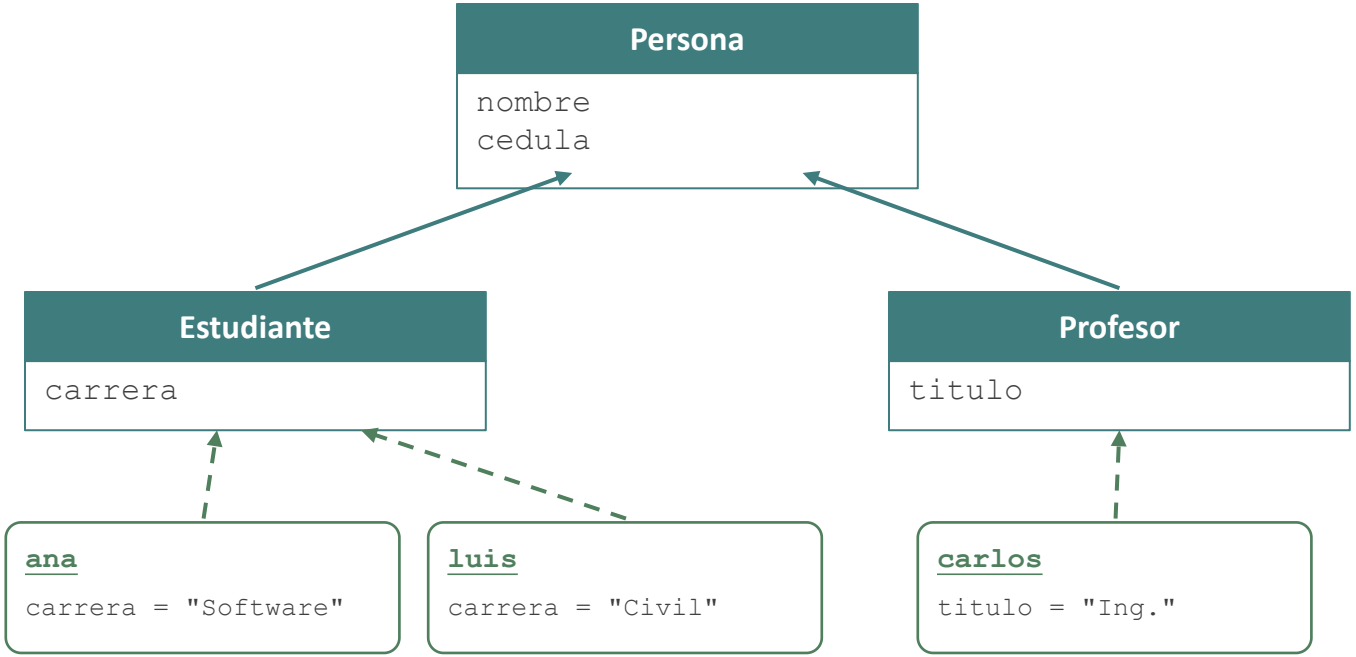
Crea la clase Figura con el método describir() y las hijas Circulo y Cuadrado que lo sobrescriban. Llama a describir() en ambas.

Clase, objeto, instancia y herencia

Concepto	Qué es	Pregunta que responde	Ejemplo
Clase	Un molde que describe atributos y métodos	¿Cómo son y qué hacen todos?	`Estudiante`
Objeto	Un elemento concreto con sus propios valores	¿Cuál es, exactamente?	Ana, 19 años, 8.7
Instancia	Un objeto visto como creado a partir de su clase	¿De qué clase salió?	ana es instancia de Estudiante
Herencia	Una relación entre dos clases, no un objeto	¿De qué clase más general viene?	Estudiante hereda de Persona

Clave: la clase y el objeto son “cosas”; la herencia no es una cosa, es la forma en que una clase se conecta con otra.

Cómo se relacionan



hereda de (clase → clase)

instancia de (objeto → clase)

ana también tiene nombre y cédula: los hereda de Persona.

Errores comunes



Confundir clase con objeto
Decir “creé la clase Ana”. Ana es un objeto; la clase es Estudiante.



Usar una referencia null
Estudiante a = null; a.nombre = "Ana"; produce NullPointerException.



Creer que c = a copia el objeto
Solo copia la referencia: los dos apuntan al mismo objeto.



Nombre de archivo distinto
public class Estudiante debe estar en Estudiante.java o no compila.



Pensar que la herencia crea objetos
La herencia une clases. Los objetos se crean con new.



Atributos public por costumbre
Sin encapsular, cualquiera puede poner un saldo negativo.

Trabajo autónomo y cierre



Trabajo autónomo

- Crea la clase Mascota con 5 atributos.
- En main, crea 2 mascotas y asígnales sus atributos.
- Antes de ejecutar, escribe la salida que esperas.
- Entrega el proyecto y la salida predicha.



Pregunta de cierre

Explica con tus palabras, sin mirar las diapositivas, la diferencia entre:

- clase y objeto
- atributo y valor
- objeto e instancia
- herencia y polimorfismo

Referencias

[1] Oracle, “Lesson: Object-Oriented Programming Concepts,” The Java Tutorials. Accedido: 14 oct. 2026. [En línea]. Disponible: https://docs.oracle.com/javase/tutorial/java/concepts/index.html

[2] Oracle, “Lesson: Classes and Objects,” The Java Tutorials. Accedido: 14 oct. 2026. [En línea]. Disponible: https://docs.oracle.com/javase/tutorial/java/javaOO/index.html

[3] Oracle, “Controlling Access to Members of a Class,” The Java Tutorials. Accedido: 14 oct. 2026. [En línea]. Disponible: https://docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html

[4] Oracle, “Inheritance,” The Java Tutorials. Accedido: 14 oct. 2026. [En línea]. Disponible: https://docs.oracle.com/javase/tutorial/java/IandI/subclasses.html

[5] Oracle, “Polymorphism,” The Java Tutorials. Accedido: 14 oct. 2026. [En línea]. Disponible: https://docs.oracle.com/javase/tutorial/java/IandI/polymorphism.html

[6] P. Deitel y H. Deitel, Cómo programar en Java, 10.ª ed. Ciudad de México, México: Pearson Educación, 2016.

[7] J. Sorva, “Notional machines and introductory programming education,” ACM Trans. Comput. Educ., vol. 13, no. 2, art. 8, pp. 1–31, jul. 2013, doi: 10.1145/2483710.2483713.